DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 – CO3 (ANSWERED)

Divide and Conquer: Design, Implementation and Analysis

Course: Design and Analysis of Algorithms

Student A: Suvedhan G Register No.: 192524381

Student B: Divya Prashanth S Register No.: 192521361

Note: This is a worked reference solution. Run the included programs on your own machine and enter actual timings where requested. The contribution record and independent design proposals must reflect the pair's genuine work.

Q1 – Decomposition and Elimination in Ordered Data (6 Marks)

Part A: Finding Maximum and Minimum

Divide-and-conquer strategy: split the array into two halves, find the minimum and maximum of each half recursively, then combine the results by taking the smaller of the two minima and the larger of the two maxima. For one element, that element is both minimum and maximum. For two elements, one comparison determines both.

Pseudocode

```
ALGORITHM MaxMin(A, low, high)
INPUT: A, an array of n values; low and high, current boundaries
OUTPUT: (minimum, maximum)

BEGIN
  IF low = high THEN
    RETURN (A[low], A[low])
  END IF

  IF high = low + 1 THEN
    IF A[low] < A[high] THEN
      RETURN (A[low], A[high])
```

```
        ELSE
            RETURN (A[high], A[low])
        END IF
    END IF

    mid ← FLOOR((low + high) / 2)
    (min1, max1) ← MaxMin(A, low, mid)
    (min2, max2) ← MaxMin(A, mid + 1, high)
    RETURN (MIN(min1, min2), MAX(max1, max2))
END
```

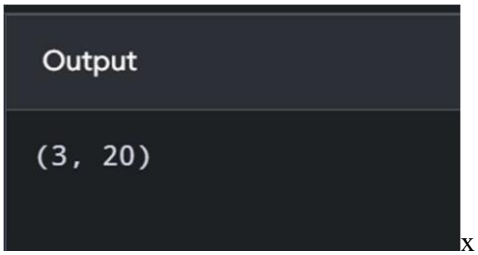
Decomposition sketch: A[low..high] → left half and right half → (min1, max1) and (min2, max2) → global minimum = MIN(min1, min2), global maximum = MAX(max1, max2).

Comparison analysis: for n = 2^k, the paired tournament method uses about 3n/2 − 2 comparisons. The recurrence for even-sized groups is T(n) = 2T(n/2) + 2 for the combine step, with base T(2) = 1, yielding T(n) = 3n/2 − 2. Finding maximum and minimum independently uses (n−1) + (n−1) = 2n−2 comparisons. The tournament method therefore saves approximately n/2 comparisons.

Python implementation with comparison counter

```
def minmax(a, l, h):
    if l == h:
        return a[l], a[l]
if h == l + 1:
        return min(a[l], a[h]), max(a[l], a[h])
    m = (l + h) // 2
    x, y = minmax(a, l, m)
    p, q = minmax(a, m + 1, h)
return min(x, p), max(y, q)
a = [12, 5, 8, 20, 3, 15]
```

print(minmax(a, 0, len(a)-1))



x

Experimental validation

Test arrays should include ascending, descending, random, all-equal, negative, and single-element cases. Increase n (for example 10, 100, 1,000, and 10,000), record the comparison counter, and compare it with the theoretical trend. For non-power-of-two sizes, exact counts depend on how the array is divided; the measured count should still grow linearly.

Dataset	Expected result / check	Purpose
[4, 1, 8, -2, 5]	minimum = -2, maximum = 8	General values
[1, 2, 3, 4, 5]	minimum = 1, maximum = 5	Ascending
[9, 7, 5, 3]	minimum = 3, maximum = 9	Descending
[6, 6, 6]	minimum = maximum = 6	Duplicates
[42]	minimum = maximum = 42	Base case

Part B: Binary Search

Binary search requires a sorted array. At each step, compare the target with the middle element. If equal, return its position. If the target is smaller, discard the right half; if larger, discard the left half. The discarded half cannot contain the target because the array is ordered.

```
ALGORITHM BinarySearch(A, target)
INPUT: A, sorted array; target, value to locate
OUTPUT: Index of target, or NOT_FOUND

BEGIN
  low ← 0
  high ← LENGTH(A) - 1
  WHILE low ≤ high DO
    mid ← FLOOR((low + high) / 2)
    IF A[mid] = target THEN
```

```
            RETURN mid
        ELSE IF target < A[mid] THEN
            high ← mid - 1
        ELSE
            low ← mid + 1
        END IF
    END WHILE
    RETURN NOT_FOUND
END
```

Termination occurs when low > high, meaning no candidate positions remain. Each unsuccessful comparison removes about half the remaining candidates, so time complexity is O(log n), with O(1) extra space for the iterative version.

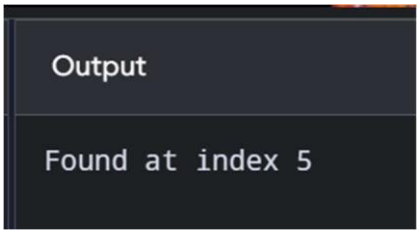
```
a = [10, 20, 30, 40, 50, 60, 70]

target = 60


l, h = 0, len(a)-1


while l <= h:
    m = (l+h)//2
    if a[m] == target:
        print("Found at index", m)
        break
    elif target < a[m]:
        h = m-1
    else:
        l = m+1
```

```
else:
    print("Not Found")
```



Experimental plan: test successful targets at the beginning, middle and end; unsuccessful targets smaller than the minimum, between values, and greater than the maximum. Record comparisons for n = 100, 1,000, 10,000 and 100,000. The comparison count should grow logarithmically, not linearly.

Q2 – The Quality of Decomposition in Sorting (7 Marks)

Merge Sort

Merge Sort divides the array into two nearly equal halves, recursively sorts both halves, and merges the sorted halves. The split is predictable regardless of the input order.

```
ALGORITHM MergeSort(A)
INPUT: Array A
OUTPUT: A sorted in ascending order

BEGIN
    IF LENGTH(A) ≤ 1 THEN
        RETURN A
    END IF
    mid ← FLOOR(LENGTH(A) / 2)
    left ← MergeSort(A[0..mid-1])
    right ← MergeSort(A[mid..LENGTH(A)-1])
    RETURN Merge(left, right)
END
def merge_sort(a):
    if len(a) <= 1:
        return list(a)
```

```
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out
```

Quick Sort and pivot strategy

Use the last element as the pivot and Lomuto partitioning for a clearly defined experiment. Quick Sort partitions values into those ≤ pivot and those > pivot, then recursively sorts the partitions. If the pivot repeatedly falls near the median, partitions are balanced and recursion depth is O(log n). If the pivot is repeatedly the smallest or largest element, partition sizes become 0 and n−1, recursion depth becomes O(n), and total time can reach $O(n^2)$.

```
def quick_sort(a):
    a = list(a)
    max_depth = 0
    partition_sizes = []

    def sort(lo, hi, depth):
        nonlocal max_depth
        max_depth = max(max_depth, depth)
        if lo >= hi:
            return
        pivot = a[hi]              # last-element pivot
        i = lo
        for j in range(lo, hi):
            if a[j] <= pivot:
                a[i], a[j] = a[j], a[i]
```

```
            i += 1
        a[i], a[hi] = a[hi], a[i]
        partition_sizes.append((i-lo, hi-i))
        sort(lo, i-1, depth+1)
        sort(i+1, hi, depth+1)

    sort(0, len(a)-1, 1)
    return a, max_depth, partition_sizes
```

Experiment design: compare random arrays, ascending arrays, descending arrays, and arrays with many duplicate values. Record partition sizes and maximum recursion depth. With a last-element pivot, already sorted input commonly creates highly unbalanced partitions. Randomised or median-of-three pivot selection reduces the chance of repeated bad partitions but does not change Quick Sort's $O(n^2)$ worst-case bound unless an introspective fallback is used.

Algorithm / case	Best / average time	Worst time	Extra space (typical)
Merge Sort	O(n log n)	O(n log n)	O(n) for array merge
Quick Sort, balanced partitions	O(n log n)	—	O(log n) recursion
Quick Sort, repeatedly unbalanced	—	$O(n^2)$	O(n) recursion

Joint design question

Before choosing, determine typical input size, whether data is nearly sorted, whether stable ordering is required, memory limits, duplicate frequency, latency requirements, and whether worst-case guarantees matter. Merge Sort is predictable and stable but needs additional memory for arrays. Quick Sort is often fast in-place and cache-friendly, but its performance depends on pivot quality. If input characteristics are unknown and predictable worst-case performance is important, Merge Sort or an introspective sorting implementation is safer.

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication (7 Marks)

Let x and y be n-digit integers. Split each at m digits: x = a·10^m + b and y = c·10^m + d. Conventional divide-and-conquer uses four products ac, ad, bc and bd. Karatsuba computes only three products: z2 = ac, z0 = bd, and z1 = (a+b)(c+d) − z2 − z0. Then xy = z2·10^(2m) + z1·10^m + z0.

The reduction from four recursive multiplications to three gives $T(n) = 3T(n/2) + O(n)$, assuming equal-size halves and linear-time additions. By the Master Theorem, $T(n) = O(n^{(\log_2 3)}) \approx O(n^{1.585})$, better asymptotically than $O(n^2)$ schoolbook multiplication.

```
ALGORITHM Karatsuba(x, y)
INPUT: Non-negative integers x and y
OUTPUT: Product x × y

BEGIN
   IF x or y has only one digit THEN
      RETURN x × y
   END IF
   m ← FLOOR(MAX(number_of_digits(x), number_of_digits(y)) / 2)
   BASE ← 10^m
   a ← FLOOR(x / BASE); b ← x MOD BASE
   c ← FLOOR(y / BASE); d ← y MOD BASE
   z2 ← Karatsuba(a, c)
   z0 ← Karatsuba(b, d)
   z1 ← Karatsuba(a + b, c + d) - z2 - z0
   RETURN z2 × BASE² + z1 × BASE + z0
END
```

```
def karatsuba(x, y):

   if x < 10 or y < 10:

      return x*y



   m = max(len(str(x)), len(str(y))) // 2

   p = 10**m



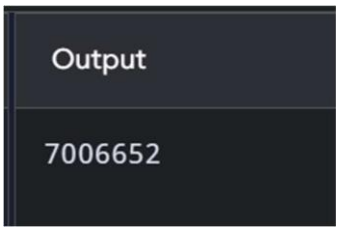
   a, b = divmod(x, p)

   c, d = divmod(y, p)
```

```
    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    adbc = karatsuba(a+b, c+d)-ac-bd

    return ac*p*p + adbc*p + bd

x, y = 1234, 5678
print(karatsuba(x, y))
```



Correctness follows by expanding the formula: z1 = ad + bc, so the reconstructed expression equals ac·10^(2m) + (ad+bc)·10^m + bd, which is the full product.

Crossover experiment: use integer operands with increasing digit lengths, repeat each test several times, and record median execution time. Verify results against schoolbook multiplication. Karatsuba may be slower for small numbers because recursion, splitting, conversions and extra additions have fixed overhead. The approximate crossover depends on the language, implementation, hardware and operand size, so it must be measured rather than assumed.

Conclusion: asymptotic complexity describes growth for large inputs; it does not include all constant factors and overheads. A theoretically faster method can lose on small inputs.

Q4 – The Reality of Strassen's Matrix Multiplication (7 Marks)

For block matrices A = [[A11,A12],[A21,A22]] and B = [[B11,B12],[B21,B22]], conventional multiplication computes eight block products. Strassen reduces these to seven products, at the cost of additional matrix additions and subtractions.

Seven products

- M1 = (A11 + A22)(B11 + B22)
- M2 = (A21 + A22)B11
- M3 = A11(B12 − B22)
- M4 = A22(B21 − B11)
- M5 = (A11 + A12)B22
- M6 = (A21 − A11)(B11 + B12)
- M7 = (A12 − A22)(B21 + B22)

Reconstruction

- C11 = M1 + M4 − M5 + M7
- C12 = M3 + M5
- C21 = M2 + M4
- C22 = M1 − M2 + M3 + M6

These expressions can be verified by expanding each Mi and cancelling terms; they recover the conventional block equations C11=A11B11+A12B21, C12=A11B12+A12B22, C21=A21B11+A22B21 and C22=A21B12+A22B22.

The recurrence is $T(n)=7T(n/2)+O(n^2)$, because seven half-size matrix multiplications are performed and matrix additions/subtractions take quadratic total work at each level. Thus $T(n)=O(n^{(\log_2 7)})\approx O(n^{2.807})$.

Padding: if the matrix order is not a power of two, extend the matrices with zero rows and columns to the next suitable size, perform multiplication, then crop the result to the original order. Padding increases work and memory, particularly when the original size is just above a power-of-two boundary.

```
def conventional_matrix_multiply(A, B):
    n, inner, cols = len(A), len(B), len(B[0])
    result = [[0] * cols for _ in range(n)]
    for i in range(n):
        for k in range(inner):
            for j in range(cols):
                result[i][j] += A[i][k] * B[k][j]
    return result
```

Experimental analysis: test several square sizes, validate Strassen output against conventional multiplication, count scalar multiplications, and measure repeated timings. Strassen is useful for sufficiently large dense matrices when additions, recursion and memory fit the environment. Conventional multiplication is often preferable for small matrices, sparse matrices, memory-constrained systems, or implementations where overhead dominates.

Q5 – Upgrading Closest Pair Using Divide and Conquer (6 Marks)

The brute-force closest-pair algorithm checks all $n(n-1)/2$ distinct point pairs and takes $O(n^2)$ time. Divide and conquer sorts points by x-coordinate, divides them at the median, recursively finds the closest pair in each half, and then checks a narrow vertical strip around the dividing line.

Pseudocode

```
ALGORITHM ClosestPair(P)
INPUT: Set P of two-dimensional points
OUTPUT: Closest pair and Euclidean distance

BEGIN
   Sort P by x-coordinate
   RETURN Closest(P)
END

FUNCTION Closest(P)
   IF LENGTH(P) ≤ 3 THEN
      RETURN brute-force closest pair in P
   END IF
   Divide P into left half L and right half R
   (pairL, dL) ← Closest(L)
   (pairR, dR) ← Closest(R)
   d ← MIN(dL, dR)
   bestPair ← pairL if dL ≤ dR, otherwise pairR
   strip ← points whose horizontal distance from split line is < d
   Sort strip by y-coordinate (or maintain y-sorted order)
   FOR each point p in strip DO
      Compare p only with following points q in y-order
```

```
        while q.y - p.y < d, subject to the geometric constant bound
        Update d and bestPair if Distance(p,q) < d
    END FOR
    RETURN (bestPair, d)
END
```

The boundary stage matters because the closest pair may straddle the split. Any cross-boundary pair shorter than d must have both points within distance d of the dividing line. In the strip, points are compared in y-order; geometric packing arguments show that only a constant number of following points need checking when the strip is organised correctly. It is not necessary to compare every point against every other point.

For O(n log n), sort once by x and maintain y-order through recursion. Sorting the strip from scratch at every recursive call can introduce an extra logarithmic factor, so implementation details matter.

```
import math


def closest(points):
    best = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i+1, len(points)):
            d = math.dist(points[i], points[j])
            if d < best:
                best = d
                pair = (points[i], points[j])

    return pair, best
```

```
p = [(2,3), (12,30), (40,50),
    (5,1), (12,10), (3,4)]

print(closest(p))
```

```
Output

(((2, 3), (3, 4)), 1.4142135623730951)
```

Experimental comparison: use the same random, clustered, grid, and near-duplicate datasets for brute force and D&C. Count exact distance evaluations and time each method repeatedly. Brute force is simple and can be faster for very small n. D&C should show better growth as n increases; the practical crossover is machine- and implementation-dependent and should be determined from actual runs.

Q6 – Advanced Divide-and-Conquer Mini-Investigations (7 Marks)

Part A: Median of Medians and QuickSelect

Median of Medians selects a pivot with a deterministic worst-case guarantee. Divide the n elements into groups of five, find each group's median, recursively select the median of those medians, partition the full array around that pivot, and recurse only into the partition containing the k-th smallest element. A constant fraction of the elements is guaranteed to be discarded, yielding $T(n)=T(n/5)+T(7n/10)+O(n)=O(n)$.

```
ALGORITHM Select(A, k)
INPUT: Array A and 1-based rank k
OUTPUT: k-th smallest element

BEGIN
  IF LENGTH(A) ≤ 5 THEN
    SORT A
    RETURN A[k - 1]
  END IF
```

```
    Divide A into groups of at most five
    Find the median of each group
    pivot ← Select(the medians, CEILING(number_of_medians / 2))
    Partition A into values less than, equal to, and greater than pivot
    IF k ≤ LENGTH(less) THEN
      RETURN Select(less, k)
    ELSE IF k ≤ LENGTH(less) + LENGTH(equal) THEN
      RETURN pivot
    ELSE
      RETURN Select(greater, k - LENGTH(less) - LENGTH(equal))
    END IF
  END
```

QuickSelect typically has expected O(n) time with random pivots but $O(n^2)$ worst-case time. A deterministic poor pivot, such as always choosing the first element on already sorted data, can repeatedly leave a partition of size $n-1$. Randomisation makes this pattern less likely but does not remove the theoretical worst case. Median of Medians spends extra work finding a strong pivot but guarantees linear worst-case time.

```
def median_of_medians_select(values, k):
    # k is 1-based; this implementation returns the k-th smallest value.
    if not 1 <= k <= len(values):
        raise ValueError("k must be between 1 and len(values)")
    if len(values) <= 5:
        return sorted(values)[k - 1]
    groups = [values[i:i+5] for i in range(0, len(values), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = median_of_medians_select(medians, (len(medians)+1)//2)
    lows = [x for x in values if x < pivot]
    equals = [x for x in values if x == pivot]
    highs = [x for x in values if x > pivot]
    if k <= len(lows):
        return median_of_medians_select(lows, k)
    if k <= len(lows) + len(equals):
```

```
        return pivot
    return median_of_medians_select(highs, k-len(lows)-len(equals))
```

Verification answer: pivot quality is central because it determines how much input is discarded at each step. Good pivots keep recursion shallow; poor pivots can make the algorithm process almost the entire remaining array repeatedly.

Part B: Meet-in-the-Middle Subset Sum

Given n numbers and target S, split the numbers into two halves. Generate every subset sum of the first half and every subset sum of the second half. For each sum x in the first list, search for S−x in the second list. This replaces a 2^n search with approximately 2^(n/2) subset sums per half, plus sorting/searching work.

```
ALGORITHM MeetInMiddleSubsetSum(A, target)
INPUT: Array A of n integers and target sum
OUTPUT: TRUE if a subset sums to target; otherwise FALSE

BEGIN
   Split A into L and R
   leftSums ← all subset sums of L
   rightSums ← all subset sums of R
   Sort rightSums
   FOR each x in leftSums DO
      IF BinarySearch(rightSums, target - x) succeeds THEN
         RETURN TRUE
      END IF
   END FOR
   RETURN FALSE
END
from bisect import bisect_left

def subset_sums(a):
   sums = [0]
   for value in a:
      sums += [s + value for s in sums]
```

```
    return sums

def meet_in_middle_subset_sum(a, target):
    mid = len(a) // 2
    left = subset_sums(a[:mid])
    right = sorted(subset_sums(a[mid:]))
    for x in left:
        need = target - x
        i = bisect_left(right, need)
        if i < len(right) and right[i] == need:
            return True
    return False
```

Time and memory: each half has about 2^(n/2) subset sums. Sorting and searching gives approximately O(2^(n/2)·n) time in this implementation and O(2^(n/2)) memory, compared with O(2^n) time for straightforward enumeration. Meet-in-the-Middle trades memory for less computation. It is attractive when n is too large for 2^n enumeration but 2^(n/2) storage is feasible. If memory is very limited, the method may not be suitable.

Q7 – Design Divide and Conquer from First Principles (10 Marks)

The assignment leaves the Day-4 challenge problem unspecified in the supplied document. Therefore, the following is a clearly labelled example design to demonstrate the required stages. Replace it with the actual challenge problem if your instructor provided one separately.

Example challenge: Count inversions in an array

An inversion is a pair (i, j) where $i < j$ and $A[i] > A[j]$. A brute-force solution examines every pair in $O(n^2)$. A divide-and-conquer solution counts inversions in the left half, the right half, and across the halves during a merge.

Stage 1: Independent design proposals (example templates)

Student A initial proposal: split the array into two halves; base case is an array of size zero or one; recursively count inversions within each half; during the combine step, count cross-inversions while merging.

Student B initial proposal: sort the left and right halves separately, then compare each left-half value with right-half values to count cross-inversions. Refinement: use the merge process rather than nested comparisons, avoiding quadratic combine work.

Stage 2: Compare the two designs

Aspect	Student A proposal	Student B proposal / refinement
Decomposition	Split into two halves	Split into two halves
Base case	Size ≤ 1 returns zero inversions	Size ≤ 1 returns zero inversions
Combine	Count cross-inversions during merge	Initially compare halves; refine to linear merge counting
Strength	Linear combine step	Identifies why a naive combine would be too slow
Final decision	Use merge-based cross-inversion counting	Agree on O(n log n) algorithm

Stage 3: Joint final design and pseudocode

```
ALGORITHM CountInversions(A)
INPUT: Array A of n values
OUTPUT: Number of pairs (i, j) where i < j and A[i] > A[j]

BEGIN
   IF LENGTH(A) ≤ 1 THEN
      RETURN (A, 0)
   END IF
   mid ← FLOOR(LENGTH(A) / 2)
   (L, leftCount) ← CountInversions(A[0..mid-1])
   (R, rightCount) ← CountInversions(A[mid..n-1])
   (S, crossCount) ← MergeAndCount(L, R)
   RETURN (S, leftCount + rightCount + crossCount)
END

FUNCTION MergeAndCount(L, R)
   i ← 0; j ← 0; merged ← empty sequence; count ← 0
   WHILE i < LENGTH(L) AND j < LENGTH(R) DO
```

```
        IF L[i] ≤ R[j] THEN
            APPEND L[i] to merged
            i ← i + 1
        ELSE
            APPEND R[j] to merged
            count ← count + LENGTH(L) - i
            j ← j + 1
        END IF
    END WHILE
    APPEND all remaining elements of L and R
    RETURN (merged, count)
END
```

Why the cross-count works: each half is sorted. If R[j] is smaller than L[i], it is smaller than every remaining value L[i], L[i+1], …, so it forms LENGTH(L)−i inversions with the remaining left-half values.

Python implementation and tests

```
def maximum_subarray(a, l, h):
    if l == h:
        return a[l]

    m = (l+h)//2

    left = maximum_subarray(a, l, m)
    right = maximum_subarray(a, m+1, h)

    s = 0
    ls = float('-inf')

    for i in range(m, l-1, -1):
        s += a[i]
        ls = max(ls, s)

    s = 0
    rs = float('-inf')

    for i in range(m+1, h+1):
        s += a[i]
        rs = max(rs, s)
```

```
    return max(left, right, ls+rs)


a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

print(maximum_subarray(a, 0, len(a)-1))
```



Stage 4: Mathematical validation

The recurrence is T(n) = 2T(n/2) + O(n): two recursive calls process half-size arrays, and merging plus counting cross-inversions takes linear time. By the Master Theorem, T(n) = O(n log n). The algorithm also uses O(n) auxiliary memory for merge buffers.

Stage 5: Experimental validation

Test sorted, reverse-sorted, random, duplicate-heavy, empty and single-element arrays. Compare against a straightforward $O(n^2)$ baseline on small and medium arrays. Both implementations must return identical inversion counts. Count pair checks in the brute-force baseline and merge operations in the D&C implementation; the gap should grow as n increases. Record actual runtime from repeated runs and use median time.

Stage 6: Joint evaluation

- Suitable for divide and conquer: the problem splits into independent left and right subproblems and the results can be combined efficiently.
- Work reduced: the nested all-pairs search is replaced by linear work per recursion level.
- Additional costs: recursion and O(n) temporary memory are introduced.
- When D&C may not be appropriate: very small inputs, when the combine step is expensive, when subproblems overlap heavily without memoization, or when memory is severely limited.

General design rule: look for a natural decomposition, manageable base cases, subproblems that are smaller and mostly independent, a correct and efficient combine step, and a recurrence that gives a meaningful improvement over the direct method.

Contribution Record (complete honestly)

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1	Max/Min design and analysis	Binary-search experiments / assigned section	Implementation and testing
Q2	Quick Sort experiments	Merge Sort analysis / assigned section	Comparison and decision
Q3	Karatsuba derivation	Crossover experiments	Correctness and conclusion
Q4	Strassen experiments	Algebraic derivation	Implementation and verification
Q5	D&C decomposition	Comparative evidence	Implementation and testing
Q6	Median of Medians	Meet-in-the-Middle	Cross-understanding and review
Q7	Mathematical validation	Experimental validation	Joint design challenge

Important: The table above mirrors the assignment's intended primary responsibilities. Edit it if the actual work differs; do not claim contributions that were not performed.

Consolidated Complexity Summary

Problem	Algorithm	Time complexity	Extra space
Maximum/minimum	Tournament divide and conquer	O(n)	O(log n) recursion
Search	Binary Search	O(log n)	O(1) iterative
Sorting	Merge Sort	O(n log n)	O(n)
Sorting	Quick Sort average / worst	O(n log n) average; $O(n^2)$ worst	O(log n) average recursion; O(n) worst
Integer multiplication	Karatsuba	$O(n^{\log_2 3}) \approx O(n^{1.585})$	Recursive temporary storage

Matrix multiplication	Strassen	$O(n^{\log_2 7}) \approx O(n^{2.807})$	Additional matrix storage
Closest pair	Brute force / D&C	$O(n^2)$ / O(n log n)	O(1) / O(n) typical
Selection	Median of Medians	O(n) worst case	Depends on implementation
Subset Sum	Meet-in-the-Middle	About $O(2^{n/2} \cdot n)$ with sorting/searching	$O(2^{n/2})$
Inversion count	Merge-based D&C	O(n log n)	O(n)

Final submission checklist

- Check every algorithm against the pseudocode convention in the supplied handbook: INPUT, OUTPUT, BEGIN, explicit conditions/loops, and RETURN.
- Run all code snippets and test edge cases before submission.
- Measure timings on your own computer; do not substitute predicted values for observed measurements.
- Add charts/tables containing your real experimental measurements if required by your faculty.
- Both partners should be able to explain all questions, not only their primary sections.
- Replace the Q7 example if the instructor supplied a different Day-4 challenge.
- Complete student details and contribution record honestly.